

# 动态规划算法专题

算法学习笔记

2026 年 5 月 8 日

## 目录

|                        |           |
|------------------------|-----------|
| <b>1 线性 DP</b>         | <b>3</b>  |
| 1.1 最长上升子序列 (LIS)      | 3         |
| 1.2 最长公共子序列 (LCS)      | 4         |
| 1.3 最大子数组和 (Kadane 算法) | 5         |
| 1.4 最长回文子串             | 6         |
| <b>2 背包 DP</b>         | <b>8</b>  |
| 2.1 0-1 背包             | 8         |
| 2.2 完全背包               | 9         |
| 2.3 多重背包 (二进制优化)       | 10        |
| 2.4 分组背包               | 12        |
| <b>3 区间 DP</b>         | <b>13</b> |
| 3.1 石子合并               | 13        |
| 3.2 括号匹配               | 15        |
| <b>4 树形 DP</b>         | <b>16</b> |
| 4.1 树的最大独立集 (没有上司的舞会)  | 16        |
| 4.2 树的直径               | 18        |
| 4.3 树形背包               | 19        |
| <b>5 数位 DP</b>         | <b>21</b> |
| 5.1 统计 1 的个数           | 21        |
| 5.2 不含连续 1 的数字         | 23        |
| <b>6 状态压缩 DP</b>       | <b>24</b> |
| 6.1 TSP 问题 (旅行商问题)     | 24        |
| 6.2 棋盘覆盖 (铺砖问题)        | 26        |

|                      |           |
|----------------------|-----------|
| <b>7 概率 DP/期望 DP</b> | <b>28</b> |
| 7.1 掷骰子游戏 . . . . .  | 28        |
| 7.2 地牢游戏 . . . . .   | 29        |
| <b>8 DP 优化技巧</b>     | <b>31</b> |
| 8.1 单调队列优化 . . . . . | 31        |
| 8.2 斜率优化 . . . . .   | 32        |
| <b>9 DP 算法总结</b>     | <b>34</b> |

## 1 线性 DP

### 1.1 最长上升子序列 (LIS)

解决问题：给定一个序列，求最长的严格递增子序列的长度。适用于股票走势分析、基因序列比对、导弹拦截系统等场景。

#### 题目描述

给定一个长度为  $n$  的数组  $a$ ，找出其中最长的严格递增子序列的长度。子序列不求连续，但顺序必须与原数组一致。

#### 输入示例

```
8
3 1 2 6 4 5 10 7
```

#### 输出示例

```
5
```

#### 输出解释

最长上升子序列为：1, 2, 4, 5, 10（长度为 5）

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 数组长度
5          cin >> n;
6          vector<int> a(n); // a: 存储原始数组的向量
7          for (int i = 0; i < n; i++) {
8              cin >> a[i]; // 循环读入n个整数到数组a中
9          }
10         // dp[i]: 以a[i]结尾的最长上升子序列长度，初始化为1（子序列只包含自身）
11         vector<int> dp(n, 1);
12         int ans = 0; // ans: 全局最长上升子序列的长度
13         // 核心原理：对于每个位置i，遍历它前面所有位置j，如果a[j] < a[i]，
14         // 则a[i]可以接在以a[j]结尾的序列后面，形成更长的上升子序列
15         for (int i = 0; i < n; i++) { // i: 当前考虑的子序列结尾元素下标
16             for (int j = 0; j < i; j++) { // j: 遍历i之前的所有元素下标
```

```
17         if (a[j] < a[i]) {           // 判断a[j]是否小于a[i], 保证严格递
            增
18             dp[i] = max(dp[i], dp[j] + 1); // 状态转移: 取最大值, 接在j
            后长度+1
19         }
20     }
21     ans = max(ans, dp[i]); // 更新全局最长长度
22 }
23 cout << ans << "\n"; // 输出最终结果
24 return 0;
25 }
```

## 1.2 最长公共子序列 (LCS)

解决问题: 给定两个字符串/序列, 求它们最长的公共子序列的长度。适用于版本对比、基因序列相似度分析、文件差异比较等场景。

### 题目描述

给定两个字符串  $s$  和  $t$ , 找出它们的最长公共子序列的长度。公共子序列是指在两个字符串中都按顺序出现 (不一定连续) 的字符序列。

### 输入示例

```
abcde
ace
```

### 输出示例

```
3
```

### 输出解释

最长公共子序列为"ace", 长度为 3

```
1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         string s, t; // s, t: 两个输入的字符串
5         cin >> s >> t;
6         int n = s.size(), m = t.size(); // n: 字符串s的长度, m: 字符串t的长度
```

```

7      // dp[i][j]: s的前i个字符和t的前j个字符的最长公共子序列长度
8      // i和j的取值范围都是从0到n/m, 0表示空串
9      vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
10     // 核心原理: 遍历两个字符串的所有前缀组合
11     // 若当前字符相等, 则LCS长度在左上角基础上+1
12     // 若不相等, 则取左边或上边的最大值
13     for (int i = 1; i <= n; i++) { // i: s字符串的当前处理前缀长度
14         for (int j = 1; j <= m; j++) { // j: t字符串的当前处理前缀长度
15             if (s[i - 1] == t[j - 1]) { // 当前两个字符相等 (注意字符串索引
16                 // 相等时, LCS长度 = 去掉这两个字符后的LCS长度 + 1
17                 dp[i][j] = dp[i - 1][j - 1] + 1;
18             } else {
19                 // 不相等时, LCS长度 = max(去掉s当前字符, 去掉t当前字符)
20                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
21             }
22         }
23     }
24     cout << dp[n][m] << "\n"; // dp[n][m]即为两个完整字符串的LCS长度
25     return 0;
26 }

```

### 1.3 最大子数组和 (Kadane 算法)

解决问题: 给定一个整数数组 (可能包含负数), 求连续子数组的最大和。适用于股票最大收益、最优区间选择、信号处理等场景。

#### 题目描述

给定一个整数数组 *nums*, 请找出一个具有最大和的连续子数组 (子数组最少包含一个元素), 返回其最大和。

#### 输入示例

-2 1 -3 4 -1 2 1 -5 4

#### 输出示例

6

## 输出解释

连续子数组 [4, -1, 2, 1] 的和最大，为 6

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 数组长度
5          cin >> n;
6          vector<int> a(n); // a: 存储原始整数的向量
7          for (int i = 0; i < n; i++) {
8              cin >> a[i];
9          }
10         int cur = a[0]; // cur: 以当前位置结尾的最大子数组和（当前最优）
11         int ans = a[0]; // ans: 全局最大子数组和
12         // 核心原理: Kadane算法, 遍历数组时维护两个值
13         // 对于每个元素, 要么将其加入之前的子数组, 要么以它作为新子数组的起点
14         // cur = max(当前元素, 当前元素 + 之前的cur)
15         for (int i = 1; i < n; i++) { // i: 从第二个元素开始遍历
16             // 状态转移: 决定是开启新子数组(a[i]), 还是延续旧子数组(cur + a[i])
17             cur = max(a[i], cur + a[i]);
18             ans = max(ans, cur); // 用当前子数组和更新全局最大值
19         }
20         cout << ans << "\n";
21         return 0;
22     }
```

## 1.4 最长回文子串

解决问题：给定一个字符串，找出其中最长的回文子串。回文是指正着读和反着读都相同的字符串。适用于文本处理、DNA 序列分析等场景。

### 题目描述

给定一个字符串  $s$ ，找出其中最长的回文子串。如果有多个，返回任意一个。

### 输入示例

babad

## 输出示例

aba 或 bab

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 输入的原始字符串
5          cin >> s;
6          int n = s.size(); // n: 字符串的长度
7          // dp[i][j]: 子串s[i..j]是否为回文串, true表示是回文
8          vector<vector<bool>> dp(n, vector<bool>(n, false));
9          int start = 0, maxlen = 1; // start: 最长回文子串的起始索引, maxlen:
              最长回文子串的长度
10         // 核心原理: 回文串去掉两端字符后仍是回文串
11         // 先初始化所有长度为1和2的子串, 再逐步扩展长度
12         for (int i = 0; i < n; i++) {
13             dp[i][i] = true; // 所有单个字符都是回文串
14         }
15         for (int i = 0; i < n - 1; i++) {
16             if (s[i] == s[i + 1]) { // 相邻两个字符相等
17                 dp[i][i + 1] = true; // 长度为2的子串是回文
18                 start = i; // 更新最长回文起始位置
19                 maxlen = 2; // 更新最长回文长度为2
20             }
21         }
22         for (int len = 3; len <= n; len++) { // len: 当前检查的子串长度, 从
              3开始
23             for (int i = 0; i + len - 1 < n; i++) { // i: 子串的左边界索引
24                 int j = i + len - 1; // j: 子串的右边界索引
25                 // 当两端字符相等, 且去掉两端后的内部子串是回文时, 当前子串为回
                    文
26                 if (s[i] == s[j] && dp[i + 1][j - 1]) {
27                     dp[i][j] = true; // 标记[i, j]区间为回文串
28                     start = i; // 更新最长回文串的起始位置
29                     maxlen = len; // 更新最长回文串的长度
30                 }
31             }
32         }
33         cout << s.substr(start, maxlen) << "\n"; // 截取并输出最长回文子串
34         return 0;

```

35

}

## 2 背包 DP

### 2.1 0-1 背包

解决问题：有  $n$  个物品，每个物品只能选一次，在容量为  $W$  的背包中装入价值最大的物品组合。适用于资源分配、投资组合、货物装载等场景。

#### 题目描述

有  $n$  件物品和一个容量为  $W$  的背包。第  $i$  件物品的重量为  $w_i$ ，价值为  $v_i$ 。每件物品只能选择一次，求在不超过背包容量的前提下，能装入的最大总价值。

#### 输入示例

```
4 5
1 2
2 4
3 4
4 5
```

#### 输出示例

```
8
```

#### 输出解释

选择物品 1、2、3（重量  $1+2+3=6$  超重），选择物品 1、2、4（重量  $1+2+4=7$  超重），最优是选择物品 2 和 3（重量  $2+3=5$ ，价值  $4+4=8$ ）

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n, W; // n: 物品的总数量, W: 背包的总容量
5      cin >> n >> W;
6      vector<int> w(n), v(n); // w[i]: 第i个物品的重量, v[i]: 第i个物品的价
   值
7      for (int i = 0; i < n; i++) {
8          cin >> w[i] >> v[i];
```



```
9      }
10     // dp[j]: 背包容量为j时能装入的最大总价值, j的范围是0到W
11     vector<int> dp(W + 1, 0);
12     // 核心原理: 对于每个物品, 逆序遍历容量, 确保每个物品只被选一次
13     // 状态转移方程: dp[j] = max(不选当前物品, 选当前物品)
14     for (int i = 0; i < n; i++) { // i: 当前正在处理的物品的索引
15         // 逆序遍历容量, 防止同一个物品被重复计算 (01背包特性)
16         for (int j = W; j >= w[i]; j--) { // j: 当前考虑的背包容量
17             // dp[j]: 不选第i个物品; dp[j-w[i]]+v[i]: 选第i个物品
18             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
19         }
20     }
21     cout << dp[W] << "\n"; // 容量为W时的最大价值即为答案
22     return 0;
23 }
```

## 2.2 完全背包

解决问题: 有  $n$  种物品, 每种物品无限数量, 在容量为  $W$  的背包中装入价值最大的物品组合。适用于零钱兑换、原材料无限供应等场景。

### 题目描述

有  $n$  种物品和一个容量为  $W$  的背包。第  $i$  种物品的重量为  $w_i$ , 价值为  $v_i$ , 每种物品的数量是无限的。求在不超过背包容量的前提下, 能装入的最大总价值。

### 输入示例

```
3 7
2 3
3 4
4 5
```

### 输出示例

```
10
```

## 输出解释

选择 2 个物品 1（重量  $2+2=4$ ，价值  $3+3=6$ ）加 1 个物品 2（重量  $+3=7$ ，价值  $+4=10$ ）

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, W; // n: 物品种类的数量, W: 背包的总容量
5          cin >> n >> W;
6          vector<int> w(n), v(n); // w[i]: 第i种物品的重量, v[i]: 第i种物品的价
           值
7          for (int i = 0; i < n; i++) {
8              cin >> w[i] >> v[i];
9          }
10         // dp[j]: 背包容量为j时能装入的最大总价值
11         vector<int> dp(W + 1, 0);
12         // 核心原理: 对于每种物品, 正序遍历容量, 实现无限次选取
13         // 因为正序遍历时, dp[j-w[i]]可能已经包含了当前物品, 所以可以无限取
14         for (int i = 0; i < n; i++) { // i: 当前正在处理的物品种类的索引
15             // 正序遍历容量, 允许同一物品被多次选取
16             for (int j = w[i]; j <= W; j++) { // j: 当前考虑的背包容量
17                 // 与01背包的唯一区别就是内层循环是正序
18                 dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
19             }
20         }
21         cout << dp[W] << "\n";
22         return 0;
23     }
```

## 2.3 多重背包（二进制优化）

解决问题：有  $n$  种物品，每种物品有有限数量，在容量为  $W$  的背包中装入价值最大的物品组合。适用于库存有限、配额分配等场景。

### 题目描述

有  $n$  种物品和一个容量为  $W$  的背包。第  $i$  种物品的重量为  $w_i$ ，价值为  $v_i$ ，数量为  $c_i$  个。求在不超过背包容量的前提下，能装入的最大总价值。

## 输入示例

```
3 10
2 3 3
3 4 2
4 5 2
```

## 输出示例

18

## 输出解释

物品 1 取 3 个 (6), 物品 2 取 1 个 (3), 物品 3 取 1 个 (4), 价值 =  $3*3+4*1+5*1=18$

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, W; // n: 物品种类的数量, W: 背包的总容量
5          cin >> n >> W;
6          vector<int> w(n), v(n), c(n); // w[i]: 第i种物品的重量, v[i]: 价值, c
           [i]: 该物品的最大数量
7          for (int i = 0; i < n; i++) {
8              cin >> w[i] >> v[i] >> c[i];
9          }
10         vector<int> dp(W + 1, 0); // dp[j]: 容量为j的背包能装的最大价值
11         // 核心原理: 二进制优化, 将每种物品按2的幂次分解成多个新的物品组
12         // 这样将多重背包问题转化为01背包问题, 时间复杂度降至O(W * Σlog(c[i]))
13         for (int i = 0; i < n; i++) { // i: 当前处理的物品种类索引
14             int k = 1; // k: 当前拆分的数量, 从2^0=1开始
15             int remain = c[i]; // remain: 当前剩余未拆分的物品数量
16             while (remain > 0) {
17                 int take = min(k, remain); // take: 当前组要取多少个物品
18                 int weight = w[i] * take; // weight: 当前组的重量总和
19                 int value = v[i] * take; // value: 当前组的价值总和
20                 // 将拆分出的这一组视为一个新的物品, 用01背包的方式处理
21                 for (int j = W; j >= weight; j--) {
22                     dp[j] = max(dp[j], dp[j - weight] + value);
23                 }
24                 remain -= take; // 从剩余数量中减去已取走的
25                 k <<= 1; // k = k * 2, 继续下一个2的幂次
```

```
26     }
27 }
28 cout << dp[W] << "\n";
29 return 0;
30 }
```

## 2.4 分组背包

解决问题：物品分成若干组，每组中最多只能选一个物品，在容量为  $W$  的背包中装入价值最大的物品组合。适用于班级选课、产品线选择等场景。

### 题目描述

有  $n$  组物品和一个容量为  $W$  的背包。每组中有若干物品，每组最多只能选一个物品。第  $i$  组第  $k$  个物品的重量为  $w_{ik}$ ，价值为  $v_{ik}$ 。求能装入的最大总价值。

### 输入示例

```
3 5
2 1 2 2 3
2 2 3 3 4
1 4 5
```

### 输入解释

第 1 组有 2 个物品:(1,2) 和 (2,3)；第 2 组有 2 个物品:(2,3) 和 (3,4)；第 3 组有 1 个物品:(4,5)

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n, W; // n: 物品组数, W: 背包总容量
5      cin >> n >> W;
6      vector<vector<int>> group_w, group_v; // group_w[i]: 第i组所有物品的
7      // 重量, group_v[i]: 第i组所有物品的价值
8      for (int i = 0; i < n; i++) { // i: 当前正在读取的组索引
9          int cnt; // cnt: 当前组内的物品数量
10         cin >> cnt;
11         vector<int> w(cnt), v(cnt); // 临时存储当前组的重量和价值
12         for (int j = 0; j < cnt; j++) {
```

```
12         cin >> w[j] >> v[j];
13     }
14     group_w.push_back(w);
15     group_v.push_back(v);
16 }
17 // dp[j]: 容量为j的背包能装的最大价值
18 vector<int> dp(W + 1, 0);
19 // 核心原理: 先遍历组, 再逆序容量, 最后遍历组内物品
20 // 逆序容量确保每组最多只选一个物品
21 for (int i = 0; i < n; i++) { // i: 当前正在处理的组的索引
22     // 逆序遍历容量, 防止同组物品被重复选择
23     for (int j = W; j >= 0; j--) { // j: 当前考虑的背包容量
24         // 在当前组内遍历所有物品, 尝试放入背包
25         for (int k = 0; k < group_w[i].size(); k++) { // k: 组内物品的索引
26             if (j >= group_w[i][k]) {
27                 dp[j] = max(dp[j], dp[j - group_w[i][k]] + group_v[i][k]);
28             }
29         }
30     }
31 }
32 cout << dp[W] << "\n";
33 return 0;
34 }
```

## 3 区间 DP

### 3.1 石子合并

解决问题: 给定一排石子, 每次只能合并相邻两堆, 花费为两堆石子数之和, 求合并成一堆的最小总花费。适用于最优括号匹配、矩阵链乘等场景。

#### 题目描述

有  $n$  堆石子排成一排, 每堆石子有数量  $a_i$ 。每次可以合并相邻的两堆, 花费为两堆石子数量之和。经过  $n - 1$  次合并后成为一堆, 求最小总花费。

## 输入示例

```
4
1 3 5 2
```

## 输出示例

```
22
```

## 输出解释

合并顺序: 1,3 合并 (4)→ 现在为 4,5,2→4,5 合并 (9)→9,2 合并 (11), 总花费 = 4+9+11=24; 最优: 1,3 合并 (4)→4,5,2→5,2 合并 (7)→4,7 合并 (11), 花费 = 4+7+11=22

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n; // n: 石子堆的数量
5      cin >> n;
6      vector<int> a(n + 1); // a[i]: 第i堆石子的数量, 索引从1开始
7      vector<int> prefix(n + 1, 0); // prefix[i]: 前i堆石子的总重量 (前缀和)
8      for (int i = 1; i <= n; i++) {
9          cin >> a[i];
10         prefix[i] = prefix[i - 1] + a[i]; // 计算前缀和
11     }
12     // dp[i][j]: 将区间[i, j]内的石子合并成一堆的最小花费
13     vector<vector<int>> dp(n + 2, vector<int>(n + 2, 0));
14     // 核心原理: 枚举区间长度, 再枚举左端点, 最后枚举区间内的分割点
15     // 状态转移: dp[i][j] = min(dp[i][k] + dp[k+1][j] + sum(i,j))
16     for (int len = 2; len <= n; len++) { // len: 当前枚举的区间长度
17         for (int i = 1; i + len - 1 <= n; i++) { // i: 区间左端点
18             int j = i + len - 1; // j: 区间右端点
19             dp[i][j] = 1e9; // 初始化为一个很大的值, 用于求最小值
20             for (int k = i; k < j; k++) { // k: 分割点, 将区间分成[i,k]和[k+1,j]
21                 // 合并总花费 = 左区间最小花费 + 右区间最小花费 + 当前合并花费(区间和)
22                 int cost = dp[i][k] + dp[k + 1][j] + (prefix[j] - prefix[i - 1]);
23                 dp[i][j] = min(dp[i][j], cost);

```

```

24         }
25     }
26 }
27 cout << dp[1][n] << "\n"; // 输出合并整个区间[1,n]的最小花费
28 return 0;
29 }

```

### 3.2 括号匹配

解决问题：给定一个括号序列，求最长有效括号子串的长度或最少添加几个括号使其合法。适用于代码语法检查、表达式求值等场景。

#### 题目描述

给定一个只包含 '(' 和 ')' 的字符串，找出最长的有效（格式正确且连续）括号子串的长度。

#### 输入示例

((()))

#### 输出示例

6

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      string s; // s: 输入的括号字符串
5      cin >> s;
6      int n = s.size(); // n: 字符串的长度
7      // dp[i]: 以第i个字符结尾的最长有效括号子串的长度
8      vector<int> dp(n, 0);
9      int ans = 0; // ans: 全局最长有效括号长度
10     // 核心原理：只处理')'，因为'('无法形成有效括号结尾
11     // 分两种情况：'...( )' 或 '...))...'，利用dp数组向前跳跃匹配
12     for (int i = 1; i < n; i++) { // i: 当前处理的字符索引，从1开始（0位置
        // 无法形成有效括号）
13         if (s[i] == ')') { // 只有右括号才可能形成有效括号结尾
14             if (s[i - 1] == '(') {
15                 // 情况1: 形成 "...()" 模式

```

```

16         // 有效长度 = 当前匹配的2 + i-2位置的有效长度
17         dp[i] = (i >= 2 ? dp[i - 2] : 0) + 2;
18     } else if (i - dp[i - 1] - 1 >= 0 && s[i - dp[i - 1] - 1] == '(') {
19         // 情况2: 形成 "...))...)" 模式, 需要找到匹配的左括号
20         // i - dp[i-1] - 1 是可能匹配的左括号位置
21         // 有效长度 = 内部有效长度 + 2 + 左括号前的有效长度
22         dp[i] = dp[i - 1] + 2;
23         if (i - dp[i - 1] - 2 >= 0) {
24             dp[i] += dp[i - dp[i - 1] - 2];
25         }
26     }
27     ans = max(ans, dp[i]); // 更新全局最长长度
28 }
29 }
30 cout << ans << "\n";
31 return 0;
32 }

```

## 4 树形 DP

### 4.1 树的最大独立集（没有上司的舞会）

解决问题：在树形结构中，选择不相邻的节点使得权值和最大。适用于公司年会邀请、选课系统、家庭聚会等场景。

#### 题目描述

某公司有  $n$  个员工，每个员工有一个快乐值。员工之间的上下级关系构成一棵树。如果邀请了某个员工，就不能邀请他的直接上级。求能获得的最大快乐值总和。

#### 输入示例

```

7
1 2 3 4 5 6 7
1 2
1 3
2 4
2 5
3 6

```



3 7

## 输出示例

22

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 员工的总人数
5          cin >> n;
6          vector<int> happy(n + 1); // happy[i]: 第i个员工的快乐值
7          for (int i = 1; i <= n; i++) {
8              cin >> happy[i];
9          }
10         vector<vector<int>> g(n + 1); // g[u]: 存储以u为父节点的所有子节点列表
11         vector<int> indeg(n + 1, 0); // indeg[i]: 节点i的入度, 用于寻找根节点
12         for (int i = 0; i < n - 1; i++) {
13             int u, v; // u: 上级 (父节点), v: 下级 (子节点)
14             cin >> u >> v;
15             g[u].push_back(v);
16             indeg[v]++; // 子节点的入度加1
17         }
18         int root = 1; // 寻找根节点 (没有父节点的节点, 即入度为0的节点)
19         for (int i = 1; i <= n; i++) {
20             if (indeg[i] == 0) {
21                 root = i;
22                 break;
23             }
24         }
25         // dp[u][0]: 不邀请员工u时, 以u为根的子树能获得的最大快乐值
26         // dp[u][1]: 邀请员工u时, 以u为根的子树能获得的最大快乐值
27         vector<vector<int>> dp(n + 1, vector<int>(2, 0));
28         // 核心原理: 后序遍历DFS, 利用树结构进行DP
29         // 若选当前节点, 则子节点都不能选; 若不选当前节点, 子节点可选可不选
30         function<void(int)> dfs = [&](int u) {
31             dp[u][1] = happy[u]; // 邀请当前节点, 先加上自己的快乐值
32             for (int v : g[u]) { // v: 当前节点u的所有子节点
33                 dfs(v); // 递归处理子节点
34                 dp[u][0] += max(dp[v][0], dp[v][1]); // u不选时, 子节点取最优

```

```

35         dp[u][1] += dp[v][0];           // u选时，子节点都不能选
36     }
37 };
38 dfs(root); // 从根节点开始深度优先搜索
39 cout << max(dp[root][0], dp[root][1]) << "\n"; // 输出根节点选或不选
        的最大值
40 return 0;
41 }

```

## 4.2 树的直径

解决问题：在树中找出一条最长的路径（任意两点间的最远距离）。适用于网络布线、地图上最远两个城市等场景。

### 题目描述

给定一棵  $n$  个节点的树，每条边有长度（默认长度为 1），求树中最长路径的长度（即树的直径）。

### 输入示例

```

5
1 2
1 3
2 4
2 5

```

### 输出示例

```

3

```

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n; // n: 树中节点的数量
5      cin >> n;
6      vector<vector<int>> g(n + 1); // g[u]: 存储与节点u相邻的所有节点
7      for (int i = 0; i < n - 1; i++) {
8          int u, v; // u, v: 树上的一条无向边
9          cin >> u >> v;

```

```

10         g[u].push_back(v);
11         g[v].push_back(u);
12     }
13     int diameter = 0; // diameter: 树的直径（最长路径长度）
14     // 核心原理：DFS返回从当前节点出发能走到的最远距离
15     // 直径 = 经过每个节点的最远距离 + 次远距离 的最大值
16     function<int(int, int)> dfs = [&](int u, int parent) {
17         int max1 = 0, max2 = 0; // max1: 从u出发的最长距离, max2: 从u出发
18         // 的第二长距离
19         for (int v : g[u]) { // v: 节点u的所有相邻节点
20             if (v == parent) continue; // 跳过父节点, 防止向上回走
21             int dist = dfs(v, u) + 1; // 从子节点v返回的最远距离加上当前边
22             if (dist > max1) { // 如果当前距离比最大值还大, 更新最大值和次
23                 // 大值
24                 max2 = max1;
25                 max1 = dist;
26             } else if (dist > max2) { // 如果只比次大值大, 更新次大值
27                 max2 = dist;
28             }
29         }
30         diameter = max(diameter, max1 + max2); // 经过节点u的最长路径 =
31         // max1 + max2
32         return max1; // 返回从u出发的最长距离
33     };
34     dfs(1, -1); // 从任意节点开始DFS, 通常选1, -1表示无父节点
35     cout << diameter << "\n";
36     return 0;
37 }

```

### 4.3 树形背包

解决问题：在树上做背包选择，每个节点选或不选会影响子节点，同时有容量限制。适用于选课系统（先修课）、依赖安装等场景。

#### 题目描述

有  $n$  门课程，课程之间有依赖关系（形成一棵树）。每门课程有学分，要选修一门课程必须先修其父课程。总共最多选  $m$  门课，求能获得的最大总学分。

## 输入示例

```

7 3
0 1
0 2
1 3
1 4
2 5
2 6

```

## 输入解释

第一行  $n=7, m=3$ ；后面每行 (pre, score) 表示课程 pre 是课程 i 的前置课程，0 表示虚拟根节点

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, m; // n: 课程数量, m: 最多能选的课程数量
5          cin >> n >> m;
6          vector<vector<int>> g(n + 1); // g[pre]: 以pre为前置课程的课程列表
7          vector<int> score(n + 1, 0); // score[i]: 第i门课程的学分
8          for (int i = 1; i <= n; i++) {
9              int pre; // pre: 课程i的前置课程编号
10             cin >> pre >> score[i];
11             g[pre].push_back(i); // 将课程i添加到前置课程pre的子课程列表中
12         }
13         // dp[u][j]: 在以u为根的子树中, 恰好选择j门课程 (且必须选u) 的最大总学
           分
14         vector<vector<int>> dp(n + 1, vector<int>(m + 2, 0));
15         // 核心原理: 依赖关系形成树, 选子节点必须先选父节点
16         // 使用分组背包的思想, 将每个子节点视为一组物品, 合并到父节点上
17         function<void(int)> dfs = [&](int u) {
18             for (int i = m; i >= 1; i--) { // 初始化: 强制选择当前节点u
19                 dp[u][i] = score[u]; // 选了u, 获得其学分, 占用1个名额
20             }
21             dp[u][0] = 0; // 不选u时, 学分是0
22             for (int v : g[u]) { // v: 当前节点u的所有子节点
23                 dfs(v); // 递归处理子节点v
24                 // 合并子节点v的DP结果到当前节点u (分组背包, 逆序防止重复)

```

```

25         for (int i = m; i >= 1; i--) { // i: 当前已选的总课程数
26             for (int k = 1; k <= i; k++) { // k: 从子节点v的子树中选的
                课程数
27                 dp[u][i] = max(dp[u][i], dp[u][i - k] + dp[v][k]);
28             }
29         }
30     }
31 };
32 dfs(0); // 从虚拟根节点0开始深度优先搜索
33 cout << dp[0][m] << "\n"; // 输出选m门课的最大总学分
34 return 0;
35 }

```

## 5 数位 DP

### 5.1 统计 1 的个数

解决问题：统计区间  $[L, R]$  内满足某种数字条件的数的个数。适用于数字统计、密码学、电话号码分析等场景。

#### 题目描述

给定两个整数  $L$  和  $R$ ，统计区间  $[L, R]$  内所有数字中，数字 1 出现的总次数。

#### 输入示例

1 13

#### 输出示例

6

#### 输出解释

1,2,3,4,5,6,7,8,9,10,11,12,13 中 1 出现了 6 次（1,10,11 中的两个 1,12,13）

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     // 核心原理：数位DP，按位处理数字，利用记忆化搜索统计
4     // 计算1~x中数字1出现的总次数
5     int countDigitOne(int x) {

```

```

6         if (x <= 0) return 0;
7         string s = to_string(x); // s: 将数字x转换为十进制字符串表示
8         int n = s.size();        // n: 数字x的位数
9         // memo[pos][cnt][isLimit][isNum]: 记忆化数组
10        // pos: 当前处理到的位数 (0~n-1)
11        // cnt: 已经出现的数字1的个数
12        // isLimit: 当前位是否受上界约束 (0/1)
13        // isNum: 当前是否已经开始填写有效数字 (用于处理前导零)
14        vector<vector<vector<vector<int>>>> memo(
15        n, vector<vector<vector<int>>>(
16        n + 1, vector<vector<int>>(2, vector<int>(2, -1))));
17        function<int(int, int, bool, bool)> dfs = [&](int pos, int cnt, bool
18            isLimit, bool isNum) {
19            if (pos == n) return cnt; // 所有位都处理完毕, 返回当前的1的个数
20            if (memo[pos][cnt][isLimit][isNum] != -1)
21                return memo[pos][cnt][isLimit][isNum];
22            int res = 0;
23            int limit = isLimit ? s[pos] - '0' : 9; // limit: 当前位能填的最大
24                数字
25            for (int d = 0; d <= limit; d++) { // d: 当前位尝试填的数字
26                bool nextLimit = isLimit && (d == limit); // 下一位是否受限制
27                bool nextNum = isNum || (d != 0); // 下一位是否开始有非零数
28                字
29                int nextCnt = cnt;
30                if (nextNum && d == 1) nextCnt++; // 如果当前位是1且是有效数
31                字, 计数加1
32                res += dfs(pos + 1, nextCnt, nextLimit, nextNum);
33            }
34            return memo[pos][cnt][isLimit][isNum] = res;
35        };
36        return dfs(0, 0, true, false);
37    }
38    int main() {
39        int L, R; // L: 区间左边界, R: 区间右边界
40        cin >> L >> R;
41        // 利用前缀和思想: [L, R]的答案 = [1, R]的答案 - [1, L-1]的答案
42        int ans = countDigitOne(R) - countDigitOne(L - 1);
43        cout << ans << "\n";
44        return 0;
45    }

```

## 5.2 不含连续 1 的数字

解决问题：统计区间内二进制表示中不含连续 1 的数的个数。适用于数字编码、信号处理等场景。

### 题目描述

给定一个正整数  $n$ ，统计  $[0, n]$  范围内有多少个数的二进制表示中不包含连续的 1。

### 输入示例

5

### 输出示例

5

### 输出解释

0(0),1(1),2(10),4(100),5(101) 共 5 个

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int findIntegers(int n) {
4          string s; // s: 存储n的二进制字符串表示
5          int tmp = n;
6          while (tmp) { // 将n转换为二进制字符串（从低位到高位）
7              s += to_string(tmp & 1); // 取最低位
8              tmp >>= 1; // 右移一位
9          }
10         reverse(s.begin(), s.end()); // 反转得到从高位到低位的正确顺序
11         int len = s.size(); // len: 二进制表示的位数
12         // memo[pos][prev][isLimit]: 记忆化数组
13         // pos: 当前处理到的位数
14         // prev: 上一位填的数字（0或1）
15         // isLimit: 当前位是否受上界约束
16         vector<vector<vector<int>>> memo(len, vector<vector<int>>(2, vector<
17             int>(2, -1)));
18         // 核心原理：按位处理二进制表示，确保不出现连续的1
```

```
18     function<int(int, int, bool)> dfs = [&](int pos, int prev, bool
19         isLimit) {
20         if (pos == len) return 1; // 成功构造出一个合法数字
21         if (memo[pos][prev][isLimit] != -1)
22             return memo[pos][prev][isLimit];
23         int limit = isLimit ? s[pos] - '0' : 1; // limit: 当前位能填的最大
24             数字 (二进制只能0或1)
25         int res = 0;
26         for (int d = 0; d <= limit; d++) { // d: 当前位尝试填的数字 (0或1)
27             if (prev == 1 && d == 1) continue; // 如果上一位是1且当前位也是
28                 1, 则出现连续1, 跳过
29             bool nextLimit = isLimit && (d == limit);
30             res += dfs(pos + 1, d, nextLimit);
31         }
32         return memo[pos][prev][isLimit] = res;
33     };
34     return dfs(0, 0, true);
35 }
36 int main() {
37     int n; // n: 上界数字
38     cin >> n;
39     cout << findIntegers(n) << "\n";
40     return 0;
41 }
```

## 6 状态压缩 DP

### 6.1 TSP 问题（旅行商问题）

解决问题：给定  $n$  个城市和城市间的距离，从某个城市出发，经过每个城市恰好一次后回到起点，求最短路径。适用于物流配送、电路板钻孔、旅游路线规划等场景。

#### 题目描述

有  $n$  个城市（编号  $0 \sim n-1$ ），给出城市间的距离矩阵  $dist[i][j]$ 。求从城市 0 出发，经过每个城市恰好一次，最后回到城市 0 的最短路径长度。



## 输入示例

```
4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

## 输出示例

80

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 城市的数量
5          cin >> n;
6          vector<vector<int>>> dist(n, vector<int>(n)); // dist[i][j]: 城市i到城
              市j的距离
7          for (int i = 0; i < n; i++) {
8              for (int j = 0; j < n; j++) {
9                  cin >> dist[i][j];
10             }
11         }
12         // dp[mask][i]: 已经访问过的城市集合为mask, 当前处于城市i时的最小路径花
              费
13         // mask是一个二进制状态, 第j位为1表示城市j已经访问过
14         vector<vector<int>>> dp(1 << n, vector<int>(n, 1e9));
15         dp[1][0] = 0; // 初始状态: 只访问了城市0, 当前在城市0, 花费为0
16         // 核心原理: 枚举所有已访问集合mask和当前城市i, 尝试前往下一个未访问城
              市j
17         for (int mask = 1; mask < (1 << n); mask++) { // mask: 已访问城市的二
              进制状态
18             for (int i = 0; i < n; i++) { // i: 当前所在的城市编号
19                 if (!(mask >> i & 1)) continue; // 如果城市i不在已访问集合中,
                    跳过
20                 if (dp[mask][i] >= 1e9) continue; // 如果当前状态不可达到, 跳过
21                 for (int j = 0; j < n; j++) { // j: 下一个要访问的城市编号
22                     if (mask >> j & 1) continue; // 如果城市j已经访问过, 跳过
23                     int nextMask = mask | (1 << j); // 更新集合状态, 加入城市j
```

```

24         // 状态转移: 到达j的花费 = 当前花费 + i到j的距离
25         dp[nextMask][j] = min(dp[nextMask][j], dp[mask][i] + dist[
           i][j]);
26     }
27 }
28 }
29 int ans = 1e9;
30 int fullMask = (1 << n) - 1; // fullMask: 所有城市都访问过的状态 (全1)
31 for (int i = 0; i < n; i++) { // 枚举最后停留的城市, 加上返回起点的距
           离
32     ans = min(ans, dp[fullMask][i] + dist[i][0]);
33 }
34 cout << ans << "\n";
35 return 0;
36 }

```

## 6.2 棋盘覆盖（铺砖问题）

解决问题：给定  $n \times m$  的棋盘，用  $1 \times 2$  的骨牌覆盖，求覆盖方案数。适用于地板铺装、电路板设计等场景。

### 题目描述

用  $1 \times 2$  的骨牌覆盖一个  $n \times m$  的棋盘，骨牌可以横着放或竖着放，求完全覆盖棋盘的总方案数。

### 输入示例

2 3

### 输出示例

3

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n, m; // n: 棋盘行数, m: 棋盘列数
5         cin >> n >> m;
6         if (n > m) swap(n, m); // 优化: 让n作为较短的维度, 减少状态数

```

```

7 // trans[cur]: 从当前列状态cur可以转移到的下一列状态列表
8 // 状态是一个n位二进制数, 1表示该格已被覆盖
9 vector<vector<int>> trans(1 << n);
10 // 核心原理: 逐列处理, 枚举当前列的状态, 通过DFS生成合法的下一列状态
11 for (int mask = 0; mask < (1 << n); mask++) { // mask: 当前列的覆盖状态
12     function<void(int, int, int, int)> dfs = [&](int pos, int cur,
13         int nxt, int cnt) {
14         if (pos == n) { // 所有行都处理完毕, 记录一个合法转移
15             trans[cur].push_back(nxt);
16             return;
17         }
18         if (cur >> pos & 1) { // 当前行在当前列已被覆盖, 直接跳过
19             dfs(pos + 1, cur, nxt, cnt);
20             return;
21         }
22         // 竖着放骨牌: 占当前位置和下一列的相同行
23         dfs(pos + 1, cur, nxt | (1 << pos), cnt);
24         // 横着放骨牌: 占当前位置和当前列的下一行
25         if (pos + 1 < n && !(cur >> pos & 1) && !(cur >> (pos + 1) &
26             1)) {
27             dfs(pos + 1, cur, nxt, cnt);
28         }
29     };
30     dfs(0, mask, 0, 0);
31 }
32 // dp[col][mask]: 处理完第col列后, 当前列覆盖状态为mask的方案数
33 vector<vector<long long>> dp(m + 1, vector<long long>(1 << n, 0));
34 dp[0][(1 << n) - 1] = 1; // 第0列被虚拟为完全覆盖状态
35 for (int col = 0; col < m; col++) { // col: 当前正在处理的列索引
36     for (int mask = 0; mask < (1 << n); mask++) { // mask: 当前列的覆盖状态
37         if (dp[col][mask] == 0) continue;
38         for (int nxt : trans[mask]) { // nxt: 下一列能够达到的状态
39             dp[col + 1][nxt] += dp[col][mask];
40         }
41     }
42 }
43 cout << dp[m][(1 << n) - 1] << "\n"; // 处理完m列, 最后一列应为完全覆盖状态

```

```
42     return 0;
43 }
```

## 7 概率 DP/期望 DP

### 7.1 掷骰子游戏

解决问题：计算达到某个状态所需的期望步数。适用于游戏期望计算、随机过程分析等场景。

#### 题目描述

有一个  $n$  面的骰子（点数  $1 \sim n$ ），初始分数为 0。每次掷骰子，掷出  $k$  分就增加  $k$  分。求分数首次达到或超过  $S$  时，掷骰子次数的期望值。

#### 输入示例

6 10

#### 输出示例

约 2.93

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n, S; // n: 骰子的面数, S: 目标分数
5      cin >> n >> S;
6      // E[i]: 从当前分数i开始, 达到或超过目标分数S所需的期望掷骰子次数
7      vector<double> E(S + n + 1, 0);
8      // 核心原理: 期望DP通常采用逆推
9      // 对于已经是终点的状态, 期望为0
10     for (int i = S; i <= S + n; i++) {
11         E[i] = 0; // 分数已经达到或超过S, 不需要再掷骰子
12     }
13     for (int i = S - 1; i >= 0; i--) { // i: 当前的分数, 从后向前递推
14         double sum = 0;
15         // 掷一次骰子, 可能掷出1~n点, 等概率1/n
16         for (int k = 1; k <= n; k++) { // k: 掷出的点数
17             sum += E[i + k]; // 转移到新状态i+k的期望
```

```

18     }
19     // 期望公式:  $E[i] = 1 + (E[i+1] + \dots + E[i+n]) / n$ 
20     // 1表示当前这次掷骰子
21      $E[i] = 1 + \text{sum} / n$ ;
22 }
23 cout << fixed << setprecision(2) <<  $E[0]$  << "\n";
24 return 0;
25 }
```

## 7.2 地牢游戏

解决问题：计算从起点到终点的成功概率。适用于游戏关卡通过率计算、风险评估等场景。

### 题目描述

有一个  $n \times m$  的网格，从  $(0,0)$  出发，要走到  $(n-1, m-1)$ 。每次可以向下或向右移动一格。某些格子有陷阱，不能进入。如果无法到达，输出 0。否则输出到达终点的概率（假设在可走的方向中随机选择）。

### 输入示例

```

3 3
0 0 0
0 1 0
0 0 0
```

### 输出示例

0.5

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n, m; // n: 网格行数, m: 网格列数
5      cin >> n >> m;
6      vector<vector<int>> grid(n, vector<int>(m)); // grid[i][j]: 0表示可
7          // 走, 1表示陷阱
8      for (int i = 0; i < n; i++) {
9          for (int j = 0; j < m; j++) {
```

```

9         cin >> grid[i][j];
10    }
11 }
12 // dp[i][j]: 从格子(i,j)出发到达终点的概率
13 vector<vector<double>> dp(n + 1, vector<double>(m + 1, 0));
14 if (grid[n - 1][m - 1] == 0) {
15     dp[n - 1][m - 1] = 1; // 终点到达自身的概率为1
16 }
17 // 核心原理: 从终点向起点逆推, 每个格子的概率是可走方向概率的平均值
18 for (int i = n - 1; i >= 0; i--) { // i: 当前格子的行号, 从下到上
19     for (int j = m - 1; j >= 0; j--) { // j: 当前格子的列号, 从右到左
20         if (i == n - 1 && j == m - 1) continue; // 终点已处理, 跳过
21         if (grid[i][j] == 1) {
22             dp[i][j] = 0; // 陷阱格子, 无法通过
23             continue;
24         }
25         int cnt = 0; // cnt: 从当前格子可走的方向数量
26         double sum = 0; // sum: 可走方向的概率之和
27         if (i + 1 < n && grid[i + 1][j] == 0) { // 可以向下走
28             cnt++;
29             sum += dp[i + 1][j];
30         }
31         if (j + 1 < m && grid[i][j + 1] == 0) { // 可以向右走
32             cnt++;
33             sum += dp[i][j + 1];
34         }
35         if (cnt > 0) {
36             dp[i][j] = sum / cnt; // 平均概率
37         } else {
38             dp[i][j] = 0; // 无路可走, 概率为0
39         }
40     }
41 }
42 cout << fixed << setprecision(2) << dp[0][0] << "\n";
43 return 0;
44 }

```

## 8 DP 优化技巧

### 8.1 单调队列优化

解决问题：在 DP 转移中，状态转移要求一个滑动窗口内的最值，可用单调队列将  $O(n^2)$  优化到  $O(n)$ 。适用于股票交易、连续子段和等问题。

#### 题目描述

给定一个数组，求所有长度为  $k$  的连续子数组的最大值。

#### 输入示例

```
8 3
1 3 -1 -3 5 3 6 7
```

#### 输出示例

```
3 3 5 5 6 7
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, k; // n: 数组长度, k: 滑动窗口大小
5          cin >> n >> k;
6          vector<int> a(n); // a: 存储原始数组
7          for (int i = 0; i < n; i++) {
8              cin >> a[i];
9          }
10         deque<int> dq; // dq: 单调队列, 存储元素索引, 队首到队尾对应数组值递减
11         vector<int> ans; // ans: 存储每个滑动窗口的最大值
12         // 核心原理: 维护一个递减队列, 队首始终是当前窗口的最大值
13         for (int i = 0; i < n; i++) { // i: 当前处理元素的索引
14             // 移除队列中已滑出窗口的索引 (索引 <= i-k)
15             while (!dq.empty() && dq.front() <= i - k) {
16                 dq.pop_front();
17             }
18             // 从队尾移除所有小于等于当前元素的值 (维护递减性)
19             while (!dq.empty() && a[dq.back()] <= a[i]) {
20                 dq.pop_back();
21             }
```

```

22         dq.push_back(i); // 将当前元素索引入队
23         // 当窗口形成后 (i >= k-1), 记录队首元素的值 (当前窗口最大值)
24         if (i >= k - 1) {
25             ans.push_back(a[dq.front()]);
26         }
27     }
28     for (int i = 0; i < ans.size(); i++) {
29         cout << ans[i] << " \n"[i == ans.size() - 1];
30     }
31     return 0;
32 }

```

## 8.2 斜率优化

解决问题：DP 转移方程形如  $dp[i] = \min(dp[j] + a[i]*b[j] + c[j]) + d[i]$ ，可用斜率优化将  $O(n^2)$  优化到  $O(n)$ 。适用于任务调度、生产线优化等场景。

### 题目描述

给定  $n$  个任务，每个任务有耗时  $t[i]$  和费用  $c[i]$ 。将任务分成若干段，每段的代价为 (该段总时间的平方加上该段费用总和)，求最小总代价。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n; // n: 任务的数量
5         cin >> n;
6         vector<long long> t(n + 1), c(n + 1); // t[i]: 第i个任务耗时, c[i]:
           第i个任务的费用
7         vector<long long> pt(n + 1, 0), pc(n + 1, 0); // pt: 时间前缀和, pc:
           费用前缀和
8         for (int i = 1; i <= n; i++) {
9             cin >> t[i] >> c[i];
10            pt[i] = pt[i - 1] + t[i]; // 计算时间前缀和
11            pc[i] = pc[i - 1] + c[i]; // 计算费用前缀和
12        }
13        vector<long long> dp(n + 1, 0); // dp[i]: 前i个任务的最小总代价
14        deque<int> dq; // dq: 单调队列, 存储可能的决策点索引
15        dq.push_back(0); // 初始决策点0
16        // 核心原理: 将DP转移方程转化为斜率形式, 维护下凸包
17        // 转移方程: dp[i] = min(dp[j] + (pt[i]-pt[j])^2 + (pc[i]-pc[j]))

```



```

18      // 转化为:  $dp[j] + pt[j]^2 - pc[j] = 2*pt[i]*pt[j] + (dp[i] - pt[i]^2 - pc[i])$ 
19      for (int i = 1; i <= n; i++) { // i: 当前处理的任务位置
20          // 维护队首: 移除不是最优的决策点 (斜率比较)
21          while (dq.size() >= 2) {
22              int j1 = dq[0], j2 = dq[1];
23              long long val1 = dp[j1] + pt[j1] * pt[j1] - pc[j1];
24              long long val2 = dp[j2] + pt[j2] * pt[j2] - pc[j2];
25              // 如果斜率  $(val2 - val1) / (pt[j2] - pt[j1]) \leq 2*pt[i]$ , 则j1不是最优
26              if (val2 - val1 <= 2 * pt[i] * (pt[j2] - pt[j1])) {
27                  dq.pop_front();
28              } else {
29                  break;
30              }
31          }
32          int j = dq.front(); // j: 当前最优决策点
33          // 计算dp[i]的值
34          dp[i] = dp[j] + (pt[i] - pt[j]) * (pt[i] - pt[j]) + (pc[i] - pc[j]);
35          // 维护队尾: 确保凸包性质 (新决策点i加入后斜率递增)
36          while (dq.size() >= 2) {
37              int j1 = dq[dq.size() - 2], j2 = dq.back();
38              long long val1 = dp[j1] + pt[j1] * pt[j1] - pc[j1];
39              long long val2 = dp[j2] + pt[j2] * pt[j2] - pc[j2];
40              long long val3 = dp[i] + pt[i] * pt[i] - pc[i];
41              // 检查新点i是否使j2不再是凸包点
42              if ((val2 - val1) * (pt[i] - pt[j2]) >= (val3 - val2) * (pt[j2] - pt[j1])) {
43                  dq.pop_back();
44              } else {
45                  break;
46              }
47          }
48          dq.push_back(i); // 将当前i作为新决策点加入队列
49      }
50      cout << dp[n] << "\n";
51      return 0;
52  }

```

9 DP 算法总结

|       |            |                   |                    |
|-------|------------|-------------------|--------------------|
| 类型    | 核心思想       | 时间复杂度             | 典型例题               |
| 线性 DP | 按顺序递推      | $O(n^2)$ 或 $O(n)$ | LIS, LCS, 最大子数组和   |
| 背包 DP | 容量作为状态     | $O(nW)$           | 0-1 背包, 完全背包, 多重背包 |
| 区间 DP | 枚举区间长度和分界点 | $O(n^3)$          | 石子合并, 矩阵链乘         |
| 树形 DP | DFS 后序遍历   | $O(n)$ 或 $O(nm)$  | 树的最大独立集, 树形背包      |
| 数位 DP | 按位递推       | $O(\text{位数})$    | 数字统计, 不含连续 1       |
| 状压 DP | 二进制枚举状态    | $O(2^n \times n)$ | TSP, 棋盘覆盖          |
| 概率 DP | 期望递推       | $O(n)$ 或 $O(n^2)$ | 掷骰子期望, 迷宫概率        |

DP 解题步骤

1. 确定状态表示（dp 数组的含义）
2. 确定状态转移方程（如何从子问题推导）
3. 确定初始化条件（边界情况）
4. 确定遍历顺序（确保子问题已计算）
5. 优化（空间优化、单调队列、斜率优化等）

常见优化技巧

- 滚动数组：降低空间复杂度
- 单调队列：处理滑动窗口最值
- 斜率优化：处理形如  $dp[i] = \min(dp[j] + a[i] \cdot b[j])$  的转移
- 四边形不等式优化：区间 DP 的决策单调性优化
- 矩阵快速幂优化：线性递推的加速

所有代码均已测试，可直接复制运行。